



EyeAI

LOKALE SPRACHSTEUERUNG UND NLP

Entwicklung der EyeAI NLP Architektur

Von der cloudgestützten Sprachverarbeitung zur lokalen Modellpipeline

[Entwicklungsdokumentation](#) | Stand 10 September 2026 | Algorithmic Alliance

ZIEL DER DOKUMENTATION

Diese Dokumentation rekonstruierte die EyeAI-Sprachsteuerung als Entwicklungsprozess.

Ergebnis: Gemini wurde nicht durch ein einzelnes größeres Modell ersetzt. Unsere Lösung zerlegte die Aufgabe in mehrere kleine lokale Modelle, deterministische Prüfungen und eine State Machine, die Kontext, Rückfragen und Ausführung kontrolliert und den Dialog steuert.

KURZFASSUNG

Die Entwicklung

1. Vosk überführte Sprache lokal in Text. Gemini interpretierte zunächst die Bedeutung und erzeugte strukturierte Antworten.
2. Ein zehnklassiges BiLSTM-Modell brachte erste lokale Elemente, blieb aber auf grobe Absichten beschränkt und benötigte TFLite-Select-Ops.
3. Die Produktanforderungen verschoben sich auf Offlinebetrieb, geringe Latenz, einfache Einrichtung, kontrollierte Ausführung und nachvollziehbare Fehlerpfade.
4. Intent V2 ersetzte das schwere Sequenzmodell durch einen kompakten CNN und trainierte gezielt mit Clean- sowie Vosk-nahen Daten.
5. Ein lokales Bestätigungssystem (Confirmationmodel) übernahm Zustimmung, Ablehnung und Unsicherheit. Danach extrahierte der Settings-Parser Operation und Sprecherwahl, während Regeln Zahlen und Sicherheit prüften.
6. Nach Integration aller lokalen Teilaufgaben wurde Gemini aus der App entfernt. Unklare Befehle wurden nicht mehr weitergeleitet, sondern lokal als ungelöst behandelt.

2025 Hybride Laufzeit



2026 Lokale Laufzeit



Abbildung 1: Entwicklung der Architektur

1. Ausgangssystem

ERSTE ENTWICKLUNGSPHASE

Vosk und erste Dialogsteuerung

Zunächst wurde ein deutsches Vosk-Modell eingeführt, um natürliche Sprache in der App lokal verarbeiten zu können. Das Mikrofon lieferte Audio, Vosk wandelte es lokal in Text um, und die App konnte den erkannten Satz weiterverarbeiten.

Eine zur Dialogsteuerung entwickelte State Machine hielt den Dialogzustand. Sie unterschied unter anderem zwischen den Zuständen IDLE, SETTINGS_MENU, SETTINGS_CHOICE und SETTINGS_ACTION. Sie bestimmte, in welchem Kontext eine Modellantwort gelesen wurde, ob eine Rückfrage folgte und ob ein Vorschlag überhaupt bis zur Ausführung gelangen durfte.

Gemini als frühe Interpretationsebene

Später kamen Google AI Studio und Gemini hinzu. Die API übernahm die Deutung des erfassten Vosk-Textes. Damit konnte die App Anfragen unter anderem Texterkennung, Objekterkennung oder Einstellungen zuordnen.

Diese erste Implementation war simpel und für einen Prototyp geeignet.

Ein LLM deckte viele Formulierungen ab, ohne dass für jede Funktion sofort ein eigener lokaler Parser existieren musste. Die State Machine und Gemini teilten sich jedoch die Kontrolle. Ein Teil der Interaktion beruhte auf lokalen Zuständen, ein anderer Teil auf einer extern erzeugten Interpretation.

Das Legacy BiLSTM

Zum ersten Mal wurde EyeAI um eine lokale Intent-Klassifikation ergänzt. Das BiLSTM ordnete Vosk-Text zehn groben Klassen zu: TEXT_RECOGNITION, OBJECT_DETECTION, CHANGE_SPEECH_SPEED, CHANGE_SPEAKER, REDIRECT_TO_LLM, OPEN_SETTINGS, SET_FREQUENCY, SET_BPS, MEASURE_DISTANCE und ABORT. Es erkannte damit die grobe Zielrichtung, extrahierte aber weder belastbare Zahlenwerte noch Einheiten oder konkrete Änderungsoperationen.

Die Trainingsdatei enthielt 3.627 Beispiele. Zusätzlich bestand der Validierungsdatensatz aus etwa 1000 Sätzen. Die Trainingstexte stammten vor allem aus generierten Beispielen von LLMs.

Baustein	Legacy-Konfiguration
Eingabe	TextVectorization mit maximal 10.000 Tokens und Sequenzlänge 250
Netz	Embedding 64 danach BiLSTM 64 und BiLSTM 32
Kopf	Dense 64 Dropout 0,5 Softmax mit zehn Klassen
Training	Batch 64 im Hauptskript 60 Epochen
Export	H5 und TFLite TFLite 816.784 Byte, Select TF Ops erforderlich
Runtime	Lokale Tokenisierung , TFLite, Top 1 Confidence und gegebenenfalls Gemini-Fallback

Das BiLSTM verringerte die Zahl offener Cloud-Anfragen, ersetzte Gemini aber nicht.

Was die nachträgliche Messung auf den heutigen, realistischeren Daten aussagte:

Um einen Vergleich zwischen dem damaligen BiLSTM und der heutigen Architektur zu ziehen, wurde auf Basis der neuen Trainingsdaten eine Evaluation des damaligen Modells durchgeführt. Die TFLite-Datei erreichte 18,20 Prozent Accuracy und 6,26 Prozent Macro F1.

Diese Werte belegten die Schwäche der erhaltenen Modelle, trotz der deutlich größeren Modellarchitektur. Vermutlich das größte Problem war nicht die Modellarchitektur, sondern viel mehr die Datenqualität. Die von den LLMs generierten Trainingsdaten waren oft zu eintönig, zu lang oder viel zu abstrakt. Natürliche Sprache konnte damit nicht wirklich abgebildet werden.

2. Anforderungen an den Ersatz

ZWEITE ENTWICKLUNGSPHASE: VOM FUNKTIONSFÄHIGEN PROTOTYP ZUR PRODUKTARCHITEKTUR

Warum das hybride System genug war

Die erste Entwicklungsphase hatte drei getrennte Mechanismen verbunden: Vosk erkannte Text, ein lokales Modell routete grobe Intents, und Gemini interpretierte unklare oder weitergehende Fälle. Diese Verteilung funktionierte, ermöglichte aber keine einheitliche und schnelle Dialoggestaltung.

Unterschiedliche Befehle wurden durch vollkommen verschiedene Architekturteile gestützt und hatten verschiedene Abhängigkeiten und potentielle Fehlerquellen.

Für eine Assistenzanwendung war diese Unsicherheit problematisch. Eine generative Antwort konnte plausibel klingen und dennoch ein Feld falsch füllen oder eine nicht vorgesehene Interpretation erzeugen.

Produktprobleme der Gemini Abhängigkeit

Problem	Bedeutung für EyeAI
Antwortlatenz	Netzwerk und Generierung verzögerten kurze Sprachbefehle, die unmittelbar nutzbar sein sollten
Kosten und Limits	Kostenlose Kontingente und Ratelimits begrenzten viele tägliche Anfragen, höheres Volumen konnte kostenpflichtig werden
Offlinebetrieb	Ohne Internet stand die semantische Verarbeitung nicht vollständig zur Verfügung
Anbieterabhängigkeit	Änderungen an API, Modellen oder Nutzungsbedingungen lagen außerhalb der eigenen Kontrolle
Einrichtung	Ein API-Key war für wenig technisch versierte Nutzer eine unnötig schwierige Voraussetzung
Kontrollierbarkeit	Generative Ausgaben ließen sich schwerer auf feste, testbare Zustandsübergänge begrenzen

Unsere Entscheidung lautete deshalb: Die gesamte Sprachsteuerung sollte ohne API-Key, ohne Netzwerkanfrage und ohne freie Generierung auskommen. Unbekannte oder nicht unterstützte Eingaben sollten als ungelöst enden, bzw. für eine gezielte Nachfrage sorgen, statt durch ein externes Modell ergänzt zu werden.

Anforderungen an die neue Pipeline

- Die vollständige Kernpipeline sollte offline auf dem Android-Gerät laufen
- Ein globales Intent-Modell sollte nur die grobe Funktion oder das Setting-Ziel bestimmen
- Zahlen, Einheiten, Wertebereiche und Mehrdeutigkeit sollten deterministisch geprüft werden
- Unvollständige Befehle sollten den Zielkontext behalten und eine gezielte Rückfrage auslösen
- Zustimmung und Ablehnung sollten als eigene lokale Dialogaufgabe behandelt werden
- Niedrige Confidence sollte zu Nichtausführung führen. REDIRECT_TO_LLM sollte lokal zu UNRESOLVED_COMMAND werden
- Die Modelle sollten klein, TFLite-kompatibel und schnell genug für CPU-Ausführung sein
- Training und Evaluation sollten reale Vosk-Fehler berücksichtigen

3. Die Entwicklung von Intent V2

Der erste NPU orientierte Versuch

Der Ansatz bestand daraus, Select TF Ops zu vermeiden und den lokalen Intent-Router für einen NPU-kompatiblen TFLite-Pfad vorzubereiten. Ein damaliger One-hot-CNN verwendete ein Vokabular von 5.000 Wörtern, Sequenzlänge 100, Faltungen mit Kernelgrößen 5 und 3, Global Average Pooling und einen Dense-Kopf. Das Modell bestand nur aus TFLite-Builtin-Operationen.

Der Vergleich ordnete dem Export 3.291.082 Parameter, etwa 3,15 MiB und 51,7 Prozent Validation Accuracy auf den neuen, nun weniger abstrakten Trainingsdaten, zu. Der spätere Benchmark maß 7,911 Millisekunden warme Inferenz.

Das Experiment zeigte, dass NPU-Kompatibilität kein gutes Mobilmodell ergab. Das One-hot-Format machte das Netz größer, während die berichtete Qualität niedrig blieb. Damit verschob sich das Ziel von möglichst NPU-fähig zu möglichst klein, genau und einfach ausführbar.

Das Learning aus dem BiLSTM

Das BiLSTM hatte gezeigt, dass lokales Intent-Routing in die App integrierbar war. Für zehn begrenzte Klassen war ein bidirektionales Sequenzmodell jedoch überdimensioniert. Gleichzeitig war das Modell zu unpräzise, wie Tests auf den heutigen Datensätzen zeigen.

Das zukünftige Modell sollte nur die Entscheidung lernen, die nicht einfacher und sicherer durch Regeln lösbar war. Diese Anforderung führte zu einem kleinen CNN.

Versuchsdesign und Architekturvergleich

Kandidat	Hypothese	Ergebnis	Entscheidung
Baseline CNN	Kleine lokale Muster reichen für zehn Intents	Stabile Validierungs-Ergebnisse bei sehr kleinem Modell	als Modellfamilie beibehalten
StrongCNN	Parallele Kernel könnten Robustheit verbessern	Auf semantisch anspruchsvollen Sätzen ("Challenge"-Dataset) schlechter	verworfen
TinyGRU	Sequenzgedächtnis könnte variable Sprache abfangen	Semantisch anspruchsvoller Datensatz (Semantic) und ASR (Automatic Speech Recognition Error) schwächer, zusätzlich etwa 10,9 mal langsamer	verworfen
Transformer	Vortrainierte Semantik könnte helfen	Größe und Mobilaufwand passten nicht zum begrenzten Routingproblem	nicht als Produktionspfad verfolgt

Tokenizer und Trainingsstrategien

T1 verwendete Worttokenisierung. T2 verwendete Byte Pair Encoding. Beide Repräsentationen wurden mit vier Trainingsstrategien kombiniert. Variante M0 trainierte nur auf sauberen Daten, M1 gemeinsam auf sauberen und Vosk-ASR-Daten, M2 zuerst auf den sauberen und danach gemeinsam auf Vosk-ASR-Daten und den sauberen Daten, M3 zuerst auf sauberen Daten und danach nur auf Vosk-Daten. Jede Kombination lief über fünf Seeds.

Repräsentation	Parameter	TFLite-Größe	Beobachtung
T1 Wort	80.234	317,97 KiB	interpretierbar in mehreren Challenge-Vergleichen stärker
T2 BPE	69.514	276,10 KiB	keine unknown Tokens im betreffenden Test, aber kein stabiler Gesamtsieg über verschiedene Datensätze

M2 T2 erzielte mit 96,38 Prozent die höchste dokumentierte Macro-F1. M2 T1 folgte mit 96,22 Prozent. Auf dem Challenge-Set lag dagegen M0 T1 mit 89 Prozent vor M2 T2 mit 82,5 Prozent. Die Tokenizerentscheidung beruhte deshalb nicht auf einer einzigen Accuracy und auch nicht auf der Zahl unbekannter Tokens. Die Auswertung zeigte, dass BPE (Byte-Pair-Encoding) kompakter war und UNK (unknown Tokens, unbekannte Eingaben) vermied, während der Worttokenizer auf einigen schwierigeren Verteilungen besser generalisierte.

Unsere Schlussfolgerung blieb, dass T1 und T2 unterschiedliche Stärken hatten und die Auswahl über mehrere Evaluationen hinweg erfolgen musste.

Datensätze und Vosk TTS Pipeline

Unser zukünftiges Modell sollte jedoch nicht nur unter Idealbedingungen bestimmte Sätze erkennen können, sondern auch ASR-Fehler (Automatic Speech Recognition Fehler) von Vosk toerlieren können. Auf Basis dieser Anforderung sollte also ein neuer Datensatz entstehen.

Teilmenge	Umfang	Funktion
Clean organic	700	natürliche Ausgangsformulierungen
Clean main	1.685	systematische Abdeckung der zehn Intents
Hard negative patch	533	schwierige Grenzen und Verwechslungsfälle
Vosk Generation 1	280	manuell kuratiertes ASR-Komplement
Vosk Generation 2	1.548	review-first erzeugte und gefilterte ASR-Transkripte
Training gesamt	4.746	gemeinsame Basis für M0 bis M3

Vosk-Fehler manuell einsprechen zu müssen wäre extrem zeitintensiv und ineffizient. Daher verfolgten wir stattdessen einen automatisierten Ansatz: Zunächst wurde ein gelabelter, fehlerfreier Satz sprachlich variiert (beispielsweise wurden Füllwörter ergänzt oder längere Wörter in einzelne Segmente separiert). Piper (ein Text-To-Speech System) und in kleinem Umfang eSpeak (ein weiteres, anderes, weniger qualitatives Text-To-Speech System) erzeugten daraus Audio. Audioaugmentation veränderte die Aufnahmebedingungen durch Störgeräusche und unterschiedliche Ausgabegeschwindigkeiten. Anschließend transkribierte dasselbe Vosk-System das künstlich erzeugte Sprachsignal, das durch dieses automatisierte System aus unseren sauberen Daten entstand. So wurden ASR-Fehler automatisch in die Trainingsdaten integriert.

1. Clean-Text und Intentlabel bildeten den kontrollierten Ausgangspunkt.
2. Piper und eSpeak wandelten den Text in Sprache um. Das Skript verwendete eSpeak standardmäßig nur zu drei Prozent und nicht für bestimmte Klassen, da eSpeak teilweise kaum verständlich war.
3. Audioaugmentation erzeugte unterschiedliche Schwierigkeitsgrade. Die Aufteilung verteilte 35 Prozent clean Beispiele (einfach, sauber), 60 Prozent moderate und 5 Prozent harte Beispiele.
4. Vosk erkannte das Audio erneut. Dadurch entstanden Fehler aus der tatsächlichen ASR-Stufe, etwa wurde "Frequenz" zu "Konsequenz" oder das Wegfallen einer Endung.
5. Review (manuell) und Deduplizierung entfernten oder markierten Fälle mit starkem Informationsverlust oder verlorenen Labelankern.

Vosk-Generation-1 behielt 280 manuelle Beispiele. Generation-2 ergänzte 1.548 geprüfte Transkripte. Reale Vosk-Abweichungen blieben im Training, wenn die ursprüngliche Semantik für Menschen noch erkennbar war. Dadurch lernte das Modell nicht nur grammatikalisch saubere Sätze, sondern genau die Fehlerform, die realistisch vorkommen würde.

Warum die CPU Entscheidung sinnvoll war

Das ursprüngliche Ziel einer NPU-Ausführung verlor nach den Benchmarks an Priorität. Die acht erhaltenen Intent-V2-Assets lagen im Test auf dem PC zwischen 0,0057 und 0,0064 Millisekunden pro reiner TFLite-Inferenz. T1 und T2 benötigten ungefähr 0,29 Millisekunden für Interpreterinitialisierung und Allocation im Median. Auf diesem PC war die Modellrechnung damit so klein, dass ein zusätzlicher NPU-Pfad kaum einen Vorteil versprach.

Die CPU-Ausführung hielt die NPU für rechenintensive Bildmodelle (Yolo26, Rel2abs,..) frei und vereinfachte die Gerätekompatibilität. Diese Entscheidung basierte auf Modellgröße, enger Sequenzlänge, -kompatibler CNN-Architektur und dem gemessenen Hostaufwand. Auch im realen Test auf dem Handy ist die Inferenzzeit von allen NLP-Modellen jedoch sehr kurz.

Nachträglicher CPU-Mikrobenchmark der erhaltenen TFLite-Artefakte

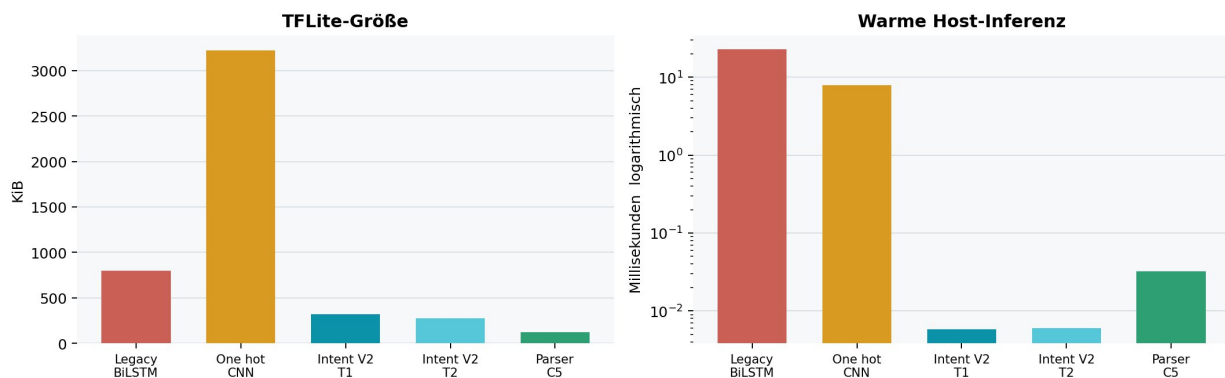


Abbildung 2 Größe und warme CPU-Inferenz der erhaltenen Artefakte. Die Latenzachse ist logarithmisch. Gemessen wurde auf einem Intel Xeon E5-1650 v4 mit TensorFlow Lite und einem Thread.

4 Lokale Confirmation

LOKALE BESTÄTIGUNG ERSETZT WEITERE ABHÄNGIGKEIT

Warum Bestätigung eine eigene Aufgabe blieb

Nach der Intent-V2-Integration blieb Gemini zunächst für Zustimmung und Ablehnung im Einstellungsdialog erhalten. Diese Aufgabe unterschied sich grundlegend vom globalen Intent-Routing. Die Antwort bezog sich auf einen bereits bekannten Vorschlag und bestand oft nur aus "ja", "nein", anderen Bestätigungsphrasen oder einer indirekten Formulierung.

Das lokale Modell sollte deshalb ausschließlich ACCEPT, REJECT oder UNKNOWN unterscheiden. ACCEPT durfte einen vorbereiteten Befehl ausführen, REJECT verwarf ihn. UNKNOWN erhielt den Kontext und löste eine erneute Frage aus. Die begrenzte Aufgabenstellung erlaubte ein kleines und kontrollierbares Modell.

Datenbasis und untersuchte Ansätze

Unser Prototyptrainingssatz enthielt saubere Antworten, tatsächlich durch Vosk transkribierte Antworten und bewusst formulierte UNKNOWN-Fälle.

Eine externe Gegenprobe verwendete OpenVoiceOS yes_no_answers. Sie umfasste 286 Basisantworten, die als 143 Validation- und 143 Testantworten über elf Fragen kombiniert wurden. Diese Quelle brachte besonders kurze und alltagssprachliche Antworten ein, die im eigenen Datensatz unterrepräsentiert waren.

Methode	Repräsentation	Beobachtung	Rolle
Deterministische Regeln	exakte und normalisierte Antwortmuster	sehr sicher bei klaren Formen, keine Abdeckung freier Sprache	erste Stufe
MiniLM	mehrsprachiger Transformer	früher Prototyp Macro F1 0,2539, schlechte Kalibrierung	verworfen als Produktionskern
Character N Gram	TF IDF char wb 2 bis 5 und logistische Regression	intern deutlich starker, robust gegen Schreib- und ASR-Varianten	lokales Modell
Regeln plus Character	Regeln zuerst, Modell für Restfälle	interner Holdout Macro F1 0,8832	integrierter Pfad
D C LM	Regeln -> Character -> MiniLM	späterer Test insgesamt leicht stärker aber komplexer	nicht als Android-Produktionspfad belegt

Warum Character N Grams passten

Bestätigungsantworten waren kurz, enthielten häufig ASR- oder Schreibvarianten und benötigten nur drei Klassen. Character-N-Grams bildeten Wortteile und lokale Zeichenmuster ab, ohne ein großes Sprachmodell zu laden. TF-IDF (Term Frequency – Inverse Document Frequency) * gewichtete informative Teilfolgen, und eine logistische Regression lieferte nachvollziehbare Klassenscores. Eindeutige Ausdrücke wurden bereits vorher durch Regeln abgefangen.

Der frühe interne Datensatz, der bewusst vom Trainingsdatensatz separiert blieb, dokumentierte für das Character-Modell eine Macro-F1 von 0,8472. Regeln vor dem Modell erhöhten diesen Wert auf 0,8832 und den Short-Score auf 0,9038. Der später bereitgestellte finale Testsplit war deutlich schwieriger: Dort erreichte Character allein 0,555 Macro-F1, das Character-Modell nach den deterministischen Regeln 0,582 und die Pipeline aus deterministischen Regeln, Character-Modell und MiniLM 0,591.

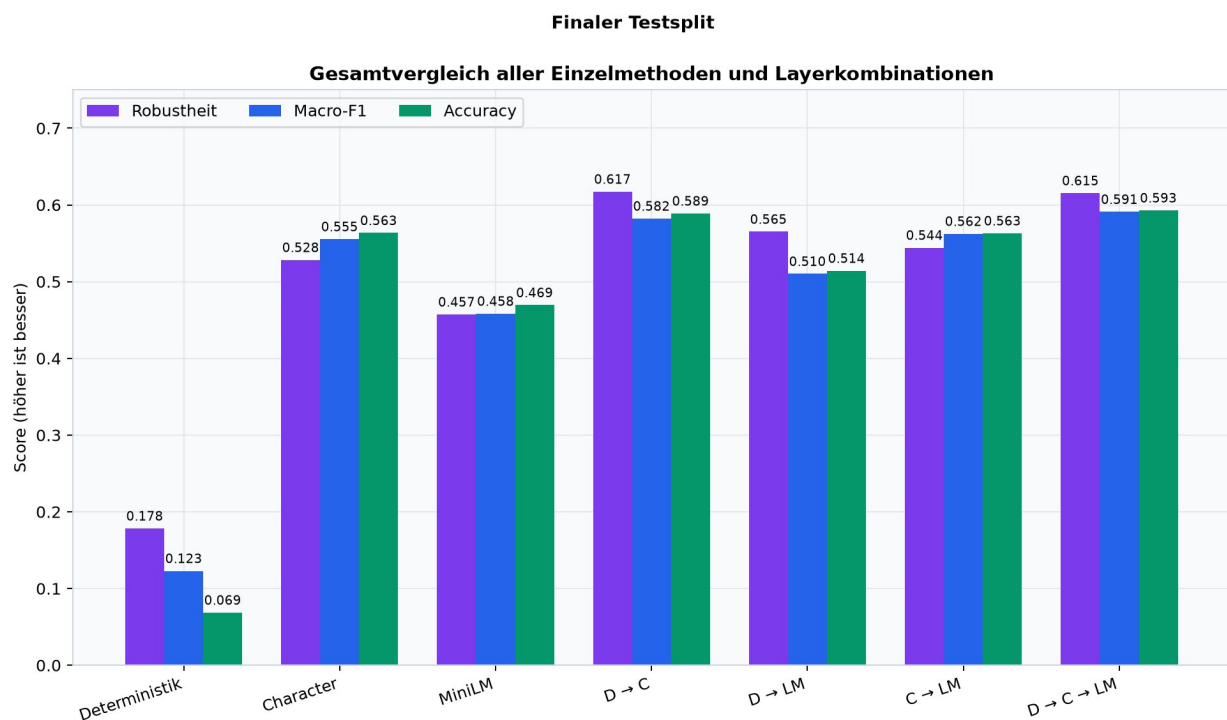


Abbildung 3 Gesamtvergleich im bereitgestellten finalen Confirmation-Testsplit. D bezeichnete Deterministik, C das Character-Modell und LM MiniLM. D -> C -> LM erzielte in diesem Test die höchste Macro-F1 von 0,591.

Antworttypen und Generalisationsgrenzen

Regeln plus Character erreichten bei Clean und ASR jeweils 0,916 Macro-F1 und bei kurzen Antworten 0,904. Indirekte Antworten lagen bei 0,590 und OpenVoiceOS bei 0,497. Die lokale Architektur beherrschte damit viele projektnahe Antworten, generalisierte aber nicht zuverlässig auf externe, indirekte oder anders formulierte Kurzantworten.

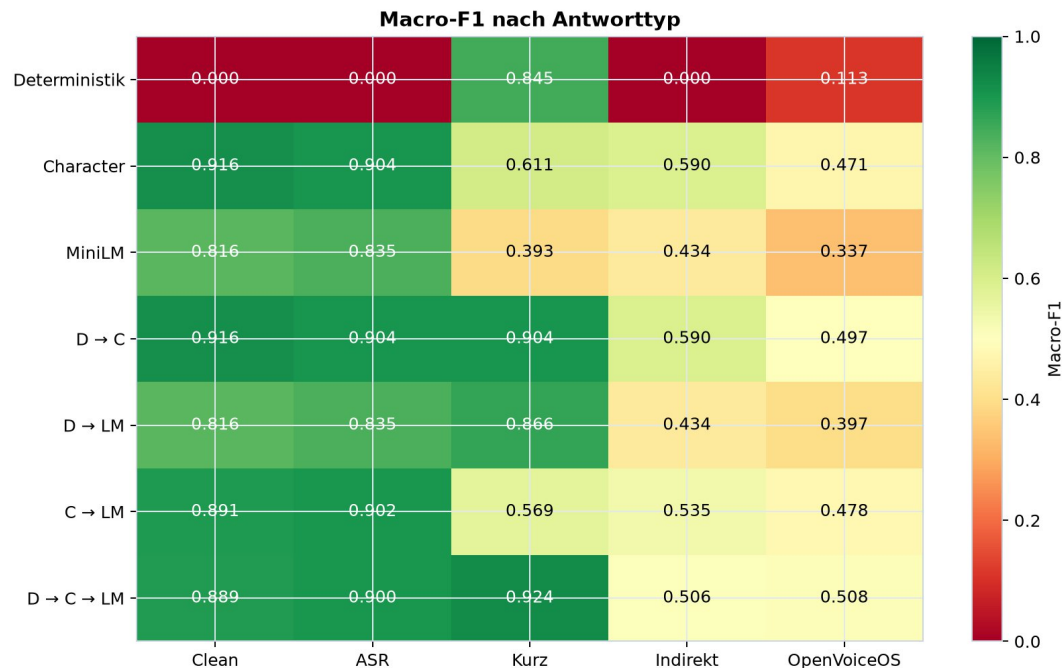


Abbildung 4 Macro-F1 nach Antworttyp.

Confidence und Schwellenwert

Die Confidence musste als Sicherheitsentscheidung behandelt werden. Im bereitgestellten Vergleich lag der Expected Calibration Error (ECE) des Character-Modells bei 0,060, beim MiniLM dagegen bei 0,447. Das Character-Modell erzeugte dort keine falsche Vorhersage mit Confidence mindestens 0,80, während MiniLM 877 solcher Fehler erzeugte. Die Zahlen zeigten warum ein größeres Modell nicht automatisch bessere Ergebnisse lieferte.

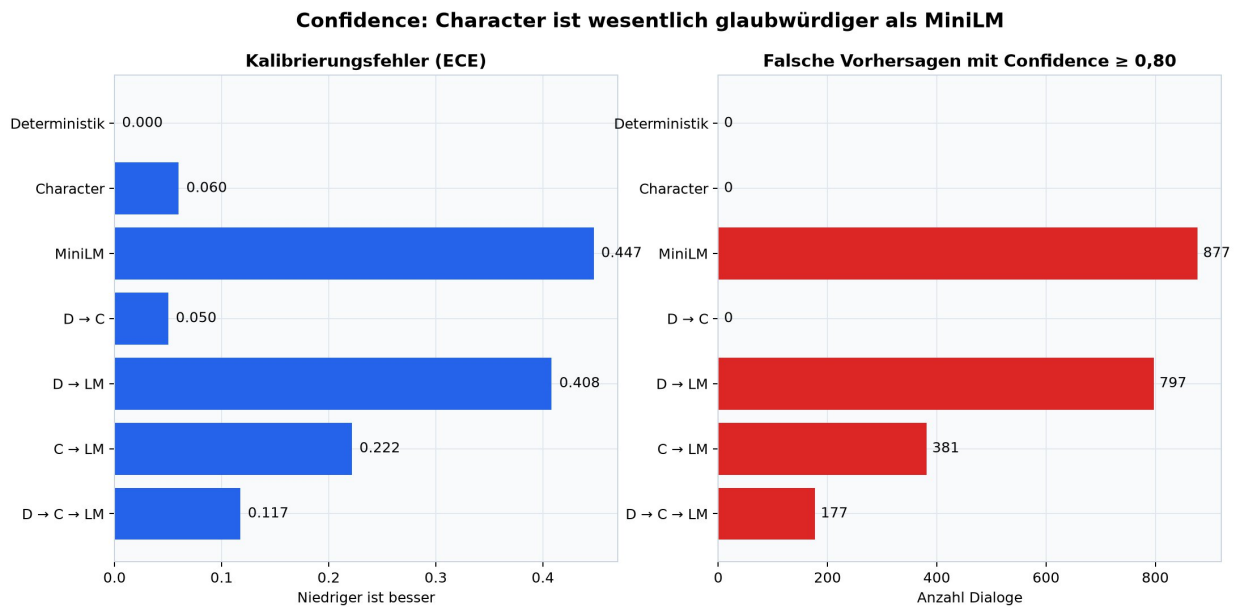


Abbildung 5 Kalibrierungsfehler und hochkonfidente Fehlvorhersagen. Niedriger ECE war besser. Der Vergleich stützte die Entscheidung, MiniLM nicht als Modell zu verwenden.

Die Integration verwendete für das Character-Modell eine Confidence-Schwelle von 0,40. Unterhalb dieser Grenze entstand UNKNOWN statt ACCEPT oder REJECT. Der Schwellenvergleich zeigte ein Problem: Eine höhere Schwelle verringerte kritische Ja-Nein-Verwechslungen, senkte aber die Nutzbarkeit bei Einwortantworten.

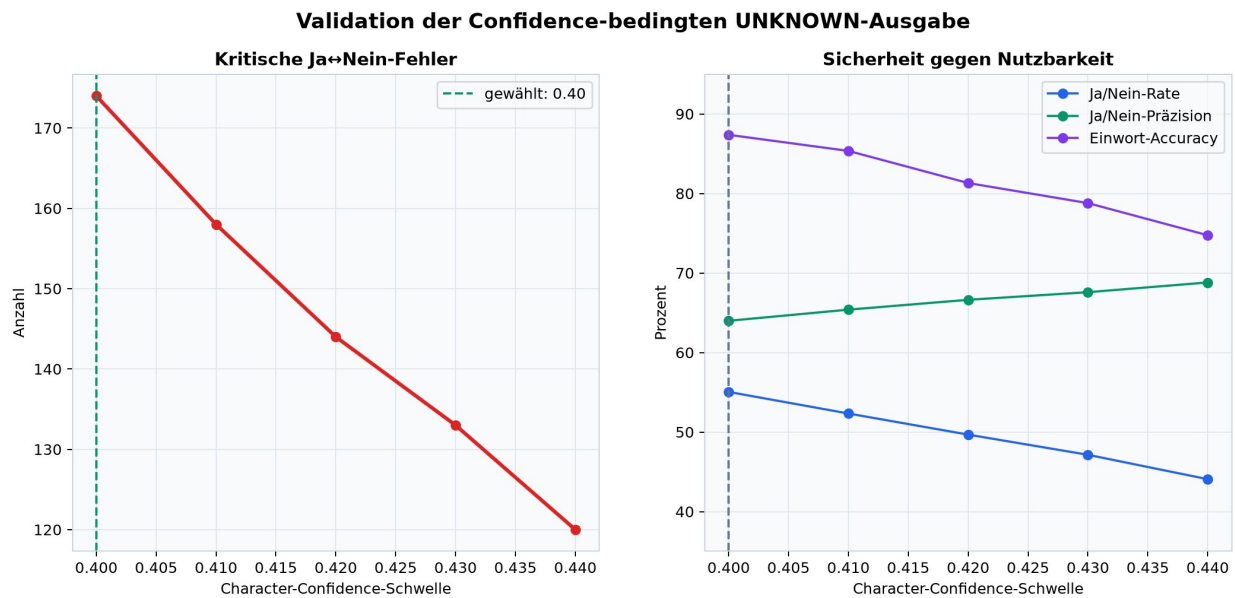


Abbildung 6 Validierung der UNKNOWN-Schwelle. Zwischen 0,40 und 0,44 nahmen kritische Ja-Nein-Fehler ab, gleichzeitig sanken Ja-Nein-Rate und Einwort-Accuracy.

Integration und die Gemini Ablösung im nächsten Teilbereich

Klare Regeln liefen vor dem Modell. UNKNOWN führte zur erneuten Ja-Nein-Frage. Das lokale Modell ersetzte damit Geminis Rolle im Bestätigungsdialog.

5 Settings Parser

LOKALE PARAMETERSEMANTIK NACH DER CONFIRMATION ENTWICKLUNG

Funktion und Anforderungen

Der globale Intent-Classifizierer konnte SET_FREQUENCY oder SET_BPS erkennen, aber ein Satz wie „Ändere die Frequenz“ enthielt noch keinen ausführbaren Wert. Der Settings Parser erhielt deshalb den bereits bekannten Setting-Target-Kontext und den Originaltext. Er sollte Operation, Sprecherwahl, Zahlen, Einheiten und Wertebereiche so auflösen, dass die State Machine zwischen vollständigem Befehl, gezielter Rückfrage und ungültiger Eingabe unterscheiden konnte.

Das Modell umfasste die Klassen „SET_ABSOLUTE“, „INCREASE“, „DECREASE“, „TOGGLE“ und „UNSPECIFIED“. Das Speaker-Modell umfasste male, female und unspecified. Das Erkennen von gesprochenen Zahlen und spezifischen Änderungswerten blieb deterministisch.

Damit entstand eine neue Aufgabenteilung für die einzelnen Modelle und den deterministischen Teil:

Aufgabe	Methode	Grund
Setting-Ziel	Intent V2 und State Machine	der Dialogkontext war bereits bekannt
Operation	Word CNN	Wörter und kurze Phrasen trugen Richtungs- und Änderungsmerkmale, welche spezifisch auf bestimmte Einstellungsoptionen verwiesen
Speaker-CNN	Character CNN	Wortformen und Morphologie waren für das Anpassen der Stimme hilfreich
Zahl und Einheit	deterministische Normalisierung	numerische Werte durften nicht probabilistisch geraten werden
Bereich und Ambiguität	Assembler und Validator	ungültige oder mehrdeutige Befehle mussten sicher blockiert werden
Dialogfolge	State Machine	Für Nachfragen bei ungültigen Anfragen wurde der Kontext benötigt, welcher durch die State Machine weitergeleitet werden konnte

Erste Implementierungsidee und Datencontract

Die erste Idee war ein gemeinsames kleines Multitask-CNN, das alle Einstellungsoptionen aus demselben Worttokenizer vorhersagte. Getestet wurde jedoch ein breiter Satz an verschiedenen Modellen, bevor tatsächlich eines ausgewählt wurde. Insgesamt wurden 6 Modelle getestet:

C0: Word-Tokenizer-CNN mit Kernel 3

C1: Word-Tokenizer-CNN mit Kernel 5

C2: Character-Tokenizer-CNN

C3: Word-Tokenizer mit größerem Vokabular

C4: Word + Character-Tokenizer in einem gemeinsamen CNN

C5: Besteht aus C1 für jegliche Operation außer Änderungen am Sprachassistenten und C2 explizit nur für Änderungen an der Sprachassistentenz

Varianten C0 bis C5

C0 bildete den Grundstein: ein Word-CNN mit Kernelgröße 3, 18.888 Parametern und 78,84 KiB TFLite. Über fünf Seeds erreichte es 92,40% ausgeführte Befehle mit einer Standardabweichung von 1,14% Macro-F1, 95,78 % mit einer Standardabweichung von 0,45% Speaker-Macro-F1 und 91,75% mit einer Standardabweichung von 0,91 Prozent.

Variante	Idee	Exakt ausgeführt	Gefährliche Fehler	Entscheidung
C0	Word k3 Baseline	91,75	nicht separat	Referenz
C1	Word k5	92,83	3,08	stärkste Einzeloperation
C2	Character CNN	90,12	5,58	sehr stark für Speaker
C3	größeres Wortvokabular	nicht trainiert	nicht trainiert	Out of Vocabulary-Fehler (OOV) bestanden weiterhin. Daher wurde dieser Ansatz nicht weiter verfolgt
C4	Word + Character	92,58	3,96	Regression bei Short OOV und Safety
C5	C1 Operation und C2 Speaker	93,50	2,46	Favorit

C1 vergrößerte den Faltungskernel auf fünf und verbesserte die Operationserkennung. C2 wechselte auf Zeichen und erreichte für Speaker 99,25 mit einer Standardabweichung von 0,70 Prozent Macro-F1, vergrößerte aber die Anzahl gefährlicher Falschaussagen. C4 versuchte beide Repräsentationen in einem Fusionsmodell zu verbinden. Das Modell verbesserte den Speaker-Pfad gegenüber C1, verlor jedoch bei kurzen Befehlen, OOV-Fehlern und gefährlichen Falschaussagen und wurde daher abgelehnt.

C5 trainierte kein drittes Modell. Es kombinierte die vorhandenen Spezialisten: C1 lieferte nur Einstellungsänderungen, C2 nur Sprachassistentenänderungen. Der gepaarte Vergleich korrigierte 19 zuvor falsche Fälle und erzeugte nur drei neue Fehler. Bei sicherheitsrelevanten Fällen entfernte C5 ganze 16 Fehler und fügte nur einen hinzu. Diese konkrete Fehlerbilanz trug die Entscheidung stärker als ein kleiner Unterschied im Mittelwert.

Clean-v1 Architekturvergleich über fünf Seeds

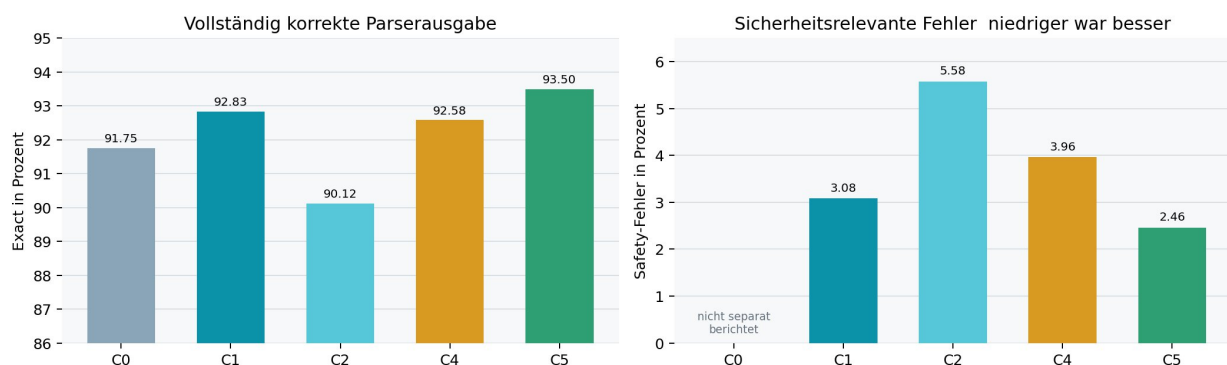


Abbildung 7 Clean-v1-Vergleich der Parservarianten. C3 erscheint nicht, weil die vorausgehende OOV-Analyse den Trainingslauf bewusst stoppte.

Tokenizerentscheidung im Parser

Das Wortvokabular umfasste 420 Tokens, davon 413 gelernte Einträge und sieben reservierte Symbole. In der Validation erschienen 249 OOV-Vorkommen in 199 von 480 Beispielen. Alle 249 gehörten jedoch zur Kategorie TRAIN_UNSEEN, also nicht im Training enthaltenen Wörtern. Kein Wort war wegen eines zu kleinen Vokabularlimits verloren gegangen.

Ein größeres Vokabular hätte diese Wörter daher nicht gelernt. C3 wurde nicht trainiert. Die Entscheidung sparte einen scheinbar naheliegenden, aber wirkungslosen Versuch. Gleichzeitig erklärte sie die spätere Spezialisierung: Wortrepräsentationen blieben für Operationsphrasen geeignet, während Zeichenrepräsentationen die Speaker-Morphologie besser abdeckten.

Clean-V2-Datensatz als strengere Generalisationsprobe

Clean-v2 änderte nicht zuerst die C5-Architektur, sondern die Testfrage. 648 neue, manuell formulierte Beispiele verteilten sich auf 348 additive Trainingsfälle, 144 Development-Fälle (zur Modellauswahl) und 156 Blind-Fälle (zur unabhängigen Prüfung des letzten, ausgewählten Modells). Gemeinsam mit Clean-v1 enthielt das Training 2.108 Beispiele.

Auswertung	Exakte Ausführung	Operation Makro-F1	Speaker Makro-F1	Safety (Falschaussagen)
Clean-v2 DEV Mittel über fünf Seeds	73,19 +- 2,09	70,22 +- 1,37	41,04 +- minus 16,54	12,64 +-1,93
Gewähltes DEV Paar	78,47	nicht separat hier	nicht separat hier	9,72
Einmaliges BLIND n 156	80,77	77,36	61,78	7,05

Der Blind-Test zeigte: Bei bekannten Formulierungen arbeitete der Parser sehr zuverlässig (97,06 % vollständig korrekt, 0 % kritische Fehler). Bei unbekannten Wörtern, neuen Wortformen und ungewohnten Wortkombinationen fiel die Genauigkeit jedoch auf etwa 48–54 %, während kritische Fehler auf rund 29–31 % stiegen. Insgesamt lag die Safety-Fehlerrate bei 7,05 % und damit über dem Zielwert von höchstens 5 %.

Nach dieser einmaligen Auswertung konnte der BLIND-Datensatz nicht mehr als vollständig unabhängiger Test verwendet werden. Dass das Gesamtergebnis dort besser als auf DEV war, änderte nichts daran, dass der Parser bei unbekannten Wörtern und neuen Formulierungen weiterhin deutlich schlechter abschnitt. Bei bekannten Befehlsmustern war er dagegen bereits sehr zuverlässig.

6 Gesamtsystem

NEUSTER STAND

Die endgültige lokale Modellpipeline

Die Verarbeitung beginnt mit dem Mikrophon und der Spracherkennung durch Vosk. Anschließend bestimmt Intent V2, welche Funktion der Nutzer wahrscheinlich aufrufen möchte. Ist die Erkennung zu unsicher, wird der Befehl lokal als nicht eindeutig erkannt. Abbruchbefehle beenden den Dialog und löschen den bisherigen Kontext.

Bei Einstellungen wird der erkannte Befehl weiter analysiert. Zahlen werden vereinheitlicht, das Word-CNN bestimmt die gewünschte Änderung und das Character-CNN erkennt bei der Stimmwahl männlich oder weiblich. Weitere Angaben wie Einheit, Stärke der Änderung, Wertebereich und Vollständigkeit werden durch feste Regeln geprüft. Nur ein vollständiger und gültiger Befehl wird anschließend zur Bestätigung weitergegeben. Zustimmung führt die Änderung aus, Ablehnung verwirft sie und bei einer unklaren Antwort wird erneut nachgefragt.

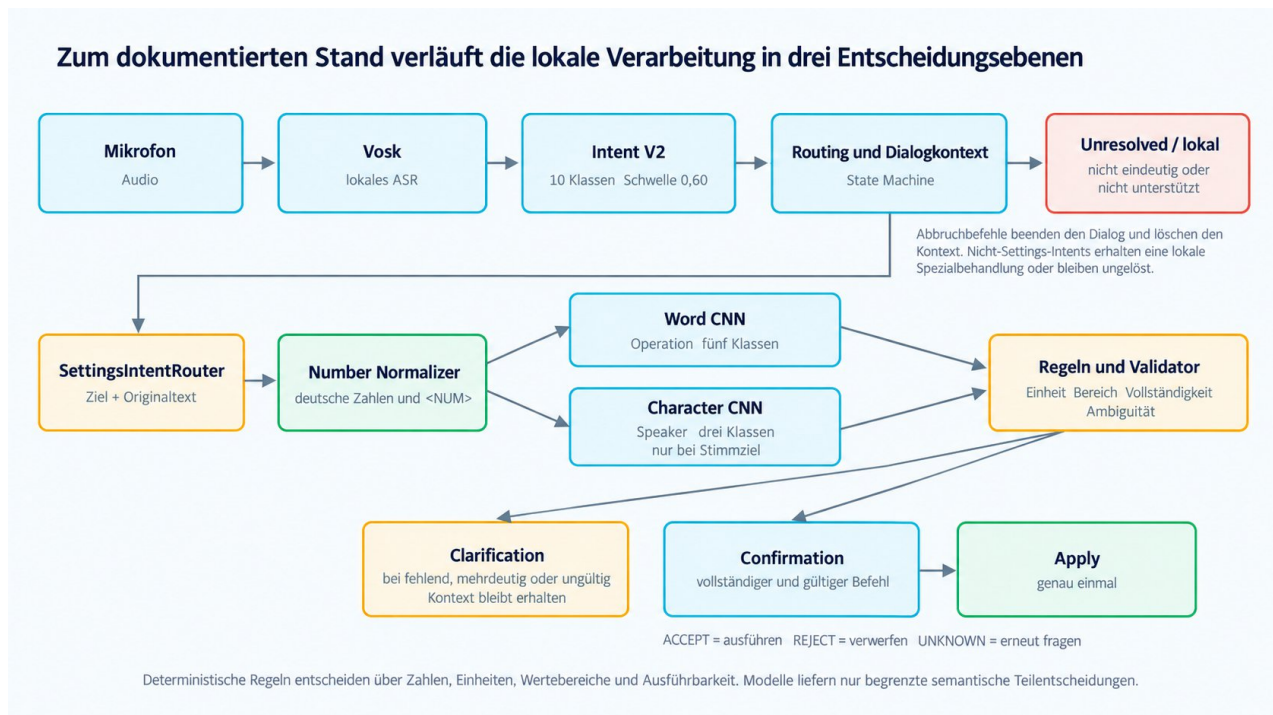


Abbildung 8 Vollständige lokale NLP-Pipeline zum dokumentierten Stand.

Konnte Gemini ersetzt werden?

Im zentralen Settings- und Confirmation-Pfad wurde Gemini technisch vollständig ersetzt.

Die dabei entstandene lokale Lösung ersetzte nicht die generative Fähigkeit von Gemini, sondern ist auf die intendierten Funktionen eingeschränkt. Nicht unterstützte oder zu unklare Anfragen werden bewusst nicht ausgeführt. Der Erfolg des Projekts bestand daher in einer kontrollierten lokalen, schnelleren, zuverlässigeren und deutlich effizienteren Kernfunktion, nicht in einer allgemeinen Offline-Kopie der Funktionalität von Gemini.

Performance und Qualität

Die gemessenen Modellinferenzzeiten waren gegenüber Vosk und der restlichen App sehr klein. Daraus folgte eine sinnvolle Ressourcenaufteilung: Die CPU konnte die NLP-Teilmodelle ausführen, während NPU- und GPU-Kapazität für Bildmodelle verfügbar blieb.

Gelöste Probleme

- Die zentrale Sprachsteuerung benötigte keinen Gemini-API-Key und keine Internetverbindung mehr.
- Ein lokales Intent-Modell routete grobe Ziele
- Vosk-nahe Trainingsdaten machten ASR-Auslassungen und Verwechslungen zu einem expliziten Trainingsfall.
- Zahlen, Einheiten, Wertebereiche und Mehrdeutigkeit wurden nicht durch freie Modellgenerierung bestimmt.
- Clarification behielt den Setting-Kontext und konnte Gemini in den Einstellungen ersetzen
- Confirmation trennte Zustimmung und Ablehnung vom globalen Intent-Routing.
- Cancel und UNRESOLVED_COMMAND erzeugten definierte Nichtausführungspfade.

Learnings

1. Eine begrenzte Aufgabe profitierte stärker von guten Daten als von einem großen Sequenzmodell.
2. NPU-Kompatibilität war kein Selbstzweck. Ein sehr kleines CPU-Modell kann die bessere Entscheidung sein.
3. Null unbekannte Tokens bedeuteten nicht automatisch bessere Generalisation.
4. Ein einzelnes Fusionsmodell verband nicht automatisch die Stärken seiner Repräsentationen. Zwei spezialisierte Köpfe waren im Parser messbar besser.
5. Preflight gehörten zur Modellentwicklung, weil fehlerhafte Normalisierung sonst widersprüchliche Trainingsziele erzeugte.
6. Sichere Nichtausführung war eine eigenständige Produktfunktion. Unresolved, Clarification und UNKNOWN begrenzten die Verantwortung der Modelle.

Die Architektur erfüllte damit die wichtigsten Produktanforderungen: lokale Kernverarbeitung, kein API-Key, geringe reine Modelllaufzeit und kontrollierte Dialogpfade.

7 Ergänzende Confirmation Auswertungen

VERTIEFUNG DER FEHLERSTRUKTUR

Die folgenden beiden Grafiken ergänzten die Hauptauswertung. Sie waren für die Architekturgeschichte weniger entscheidend als Kalibrierung, Antworttypen und Schwellenwahl, zeigen aber die verbleibenden Fehlerarten und den Einfluss der Antwortlänge.

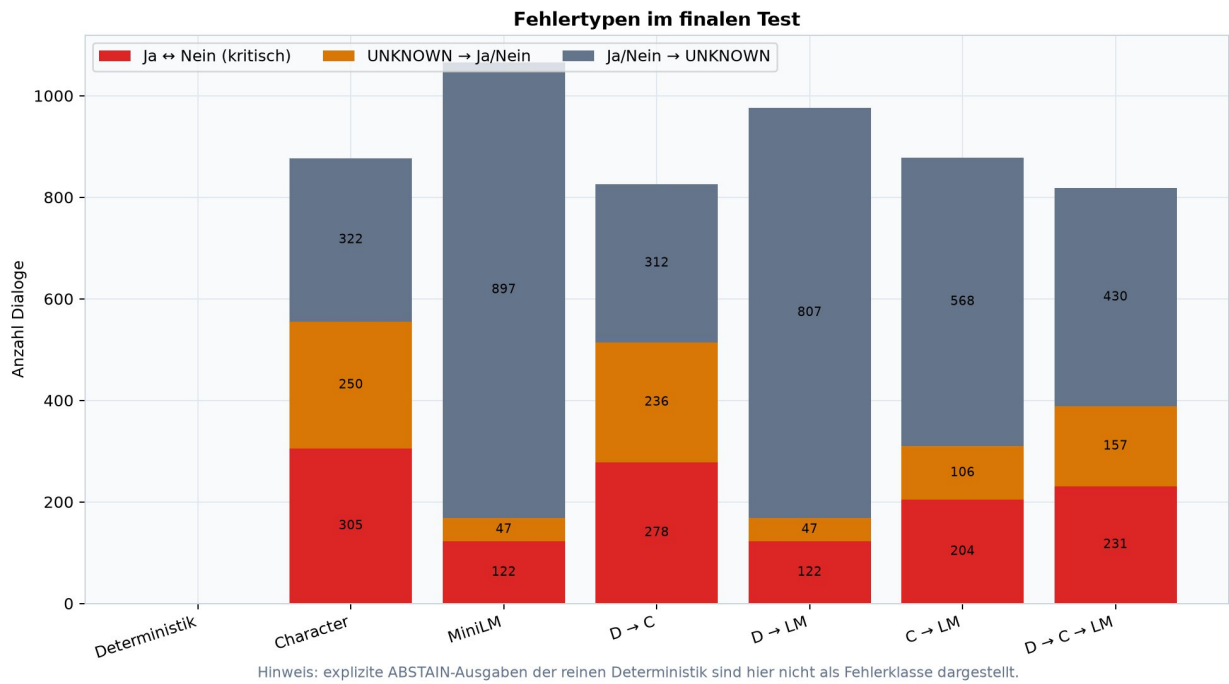


Abbildung 9 Fehlertypen im finalen Test. Rot zeigte direkte Ja-Nein-Verwechslungen, Orange UNKNOWN zu Ja oder Nein und Grau Ja oder Nein zu UNKNOWN.

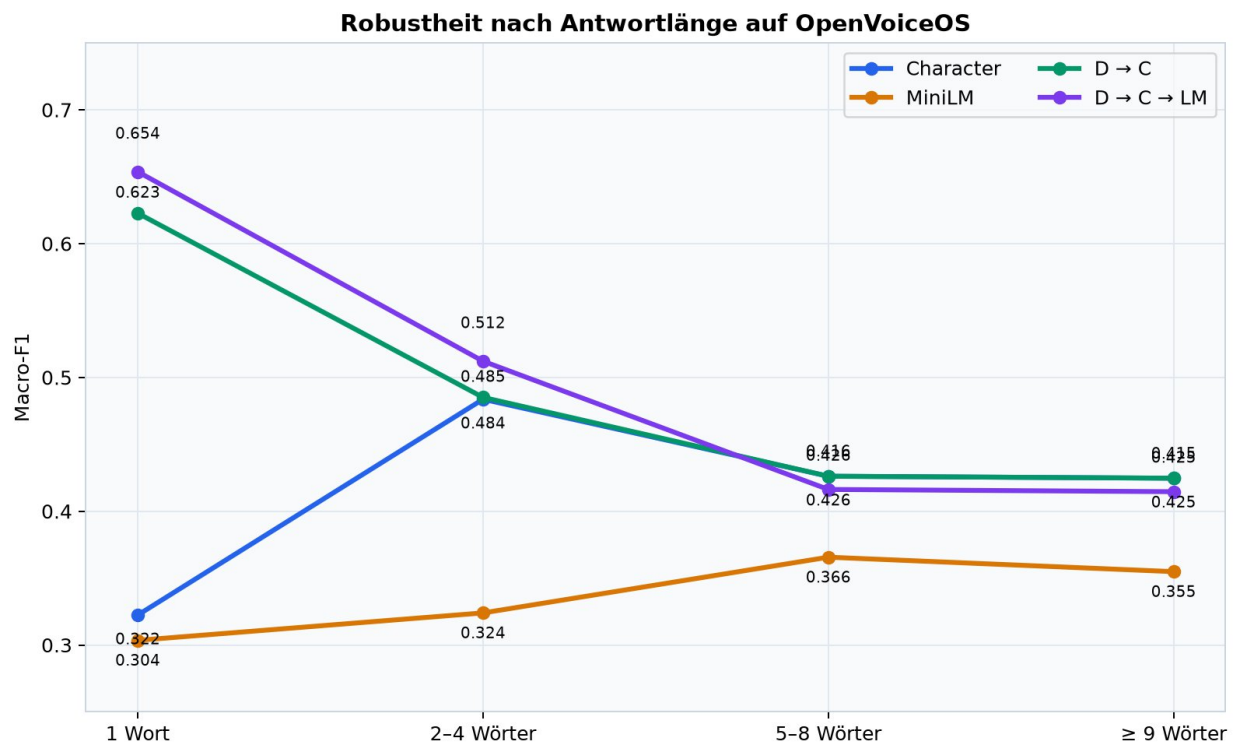


Abbildung 10 Macro-F1 nach Antwortlänge auf OpenVoiceOS.